PHP POO — Abstraction & Héritage

Semaine 3 · Mercredi · 1h · C6 — Composants d'accès aux données | C7 — Back-end

1. Concepts clés

1.1 Héritage — extends

L'héritage permet à une classe fille de réutiliser les attributs et méthodes d'une classe parent. On utilise le mot-clé extends.

Mot-clé / Concept	Rôle — Exemple
extends	Déclare l'héritage. → class Rectangle extends Forme
parent::__construct()	Appelle le constructeur de la classe parent depuis le constructeur enfant. Indispensable pour initialiser les attributs hérités.
parent::methode()	Appelle n'importe quelle méthode du parent depuis l'enfant, pour réutiliser son comportement.
Override (surcharge)	Redéfinir une méthode dans l'enfant pour remplacer ou enrichir le comportement hérité.
protected	Visibilité intermédiaire : accessible dans la classe ET ses sous-classes, mais pas depuis l'extérieur.

1.2 Classe abstraite — abstract class

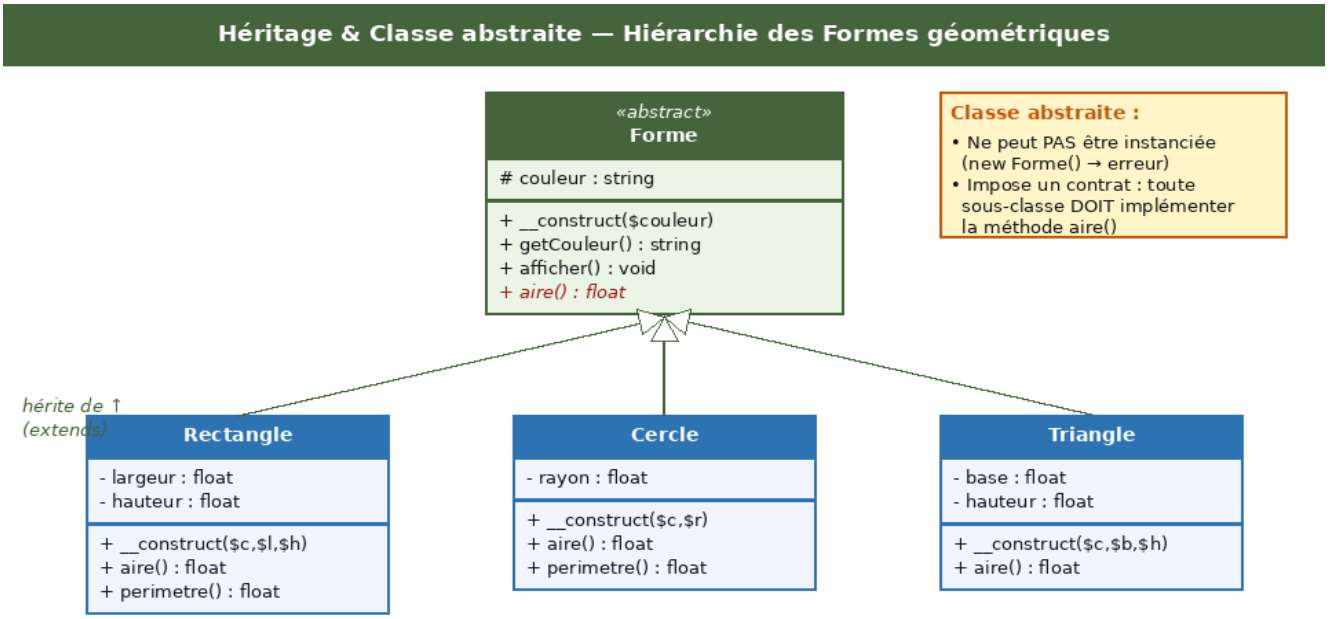
Caractéristique	Détail
abstract class	Ne peut pas être instanciée directement (new Forme() → erreur fatale). Sert de contrat/template pour les sous-classes.
abstract function	Méthode sans corps dans le parent. Toute sous-classe concrète est OBLIGÉE de l'implémenter sous peine d'erreur.
Classe concrète	Implémente toutes les méthodes abstraites héritées. Peut être instanciée avec new.

Analogie : abstract class = contrat de travail qui impose des obligations. Les sous-classes = employés qui doivent respecter le contrat.
Si une sous-classe n'implémente pas toutes les méthodes abstraites → PHP lève une erreur fatale au chargement.

2. Exemples de diagrammes & code

2.1 Hiérarchie — Forme → Rectangle, Cercle, Triangle

La classe abstraite Forme impose la méthode aire() à toutes ses sous-classes. Chaque forme implémente son propre calcul d'aire.



2.2 Code annoté — abstract class & extends

À gauche : la classe abstraite Forme avec la méthode aire() sans corps. À droite : Rectangle qui l'étend et implémente obligatoirement aire().

Code annoté — abstract class Forme & class Rectangle extends Forme

```php
<?php
abstract class Forme {

    protected $couleur;

    public function __construct($couleur) {
        $this->couleur = $couleur;
    }

    public function getCouleur() {
        return $this->couleur;
    }

    // Méthode abstraite — PAS de corps
    abstract public function aire(): float;

}
```

— abstract class : ne peut pas être instanciée directement
— protected : accessible dans les sous-classes
— abstract : pas de corps ici. Les sous-classes DOIVENT l'écrire

```php
class Rectangle extends Forme {
    private $largeur;
    private $hauteur;

    public function __construct(
            $couleur, $largeur, $hauteur) {
        parent::__construct($couleur);
        $this->largeur = $largeur;
        $this->hauteur = $hauteur;
    }

    // Implémentation OBLIGATOIRE de aire()
    public function aire(): float {
        return $this->largeur * $this->hauteur;
    }
}
```

— extends : hérite Récupère coule
— parent::__const le constructeur
— Override obliga aire() promis pa

Utilisation :

```php
$r = new Rectangle("rouge", 5, 3);
echo $r->aire();          echo $r->getCouleur();      // new Forme("bleu"); → Fatal erro
```

2.3 parent:: et surcharge de méthode

parent:: permet d'appeler le code du parent sans le réécrire. La surcharge complète (sans parent::) remplace entièrement le comportement hérité.

parent:: et surcharge de méthode (override)

Forme (parent)

```php
public function afficher() {
    echo "Forme : ";
    echo $this->couleur;
}
```

parent:: appel

Cercle (enfant)

```php
public function afficher() {
    parent::afficher();
    echo " — rayon: ";
    echo $this->rayon;
}
```

sortie

Résultat affiché :

```
Forme : rouge
 — rayon: 7
```

Surcharge complète (sans parent::) :

```php
class Triangle extends Forme {
    public function afficher() {
        echo "Triangle " . $this->couleur . " b=" . $this->base;
    }   // ← remplace ENTIÈREMENT la méthode parent
}
```

Quand utiliser parent:: ?

- Quand tu veux AJOUTER au comportement du parent
- Quand le parent fait du travail utile à réutiliser
- ⚠ Sans parent:: → comportement parent totalement remplacé

3. Exercices pratiques

✏ Exercice 1 — Gestion de projets — Guidé (30 min)

Contexte :

Une agence digitale gère des projets composés de tâches. Chaque tâche a une durée estimée. Le chef de projet veut calculer le budget total d'un projet en se basant sur les heures estimées, en ajoutant une marge de 10% pour les imprévus. Il veut aussi pouvoir lister les tâches dépassant un certain seuil d'heures.

Structure des classes :

- `class Task { private $id; private $description; private $estimatedHours; }`
- `class Project { private $title; private $dailyRate; private $tasks = []; }`
- `→ Task : constructeur + getters + méthode isBig($threshold) qui retourne true si estimatedHours > $threshold`
- `→ Project : constructeur + addTask(Task $task) + getter $tasks`
- `→ Project : calculateTotalHours() — somme des heures de toutes les tâches`
- `→ Project : calculateTotalWithBuffer($bufferPercent = 10) — total * (1 + buffer/100)`
- `→ Project : calculateBudget() — totalHeures * dailyRate / 8  (taux journalier / 8h)`
- `→ Project : getBigTasks($threshold) — retourne un tableau des tâches dont isBig() est true`

Travail demandé :

1. Créez la classe Task avec ses 3 attributs private, son constructeur, ses getters et la méthode isBig($threshold)
2. Créez la classe Project avec $title, $dailyRate (taux journalier), $tasks (tableau vide par défaut)
3. Implémentez addTask(Task $task) : ajoute la tâche au tableau $tasks
4. Implémentez calculateTotalHours() : parcourt $tasks avec foreach et additionne les heures
5. Implémentez calculateTotalWithBuffer($bufferPercent = 10) : retourne les heures + la marge en %
6. Implémentez calculateBudget() : heures totales avec buffer × (dailyRate / 8)
7. Implémentez getBigTasks($threshold) : retourne un tableau filtré des tâches dépassant le seuil
8. Testez avec un projet 'Refonte site web' (600€/jour), 3 tâches : 'Design' 12h, 'Dev front' 30h, 'Dev back' 45h
9. Affichez : total heures sans buffer, total avec 10% buffer, budget estimé, tâches > 20h

🚀 **Bonus :**

a. Ajoutez une méthode getSummary() qui retourne une chaîne formatée avec toutes les infos du projet
b. Ajoutez une méthode getMostExpensiveTask() qui retourne la tâche avec le plus d'heures

✏ Exercice 2 — Catalogue de véhicules — Semi-autonome (30 min)

Contexte :

Un concessionnaire gère différents types de véhicules : voitures, motos et camionnettes. Tous les véhicules ont une marque, un modèle, une année et un prix de base. Chaque type de véhicule calcule son prix final différemment : la voiture ajoute des frais de mise en route (150€), la moto applique une remise de 5% si l'année < 2020, la camionnette ajoute un malus écologique calculé en fonction de sa charge utile.

Structure des classes :

- `abstract class Vehicule { protected $marque; protected $modele; protected $annee; protected $prixBase; }`
- `class Voiture extends Vehicule { private $fraisMiseEnRoute = 150; }`
- `class Moto extends Vehicule { private $remiseAncienne = 0.05; }`
- `class Camionnette extends Vehicule { private $chargeUtile; }`
- → `abstract function getPrixFinal() : float` (obligatoire dans chaque sous-classe)
- → `abstract function getDescription() : string`

Travail demandé :

10. Créez la classe abstraite Vehicule avec ses 4 attributs protected, son constructeur, ses getters
11. Déclarez les méthodes abstraites getPrixFinal() et getDescription() dans Vehicule
12. Créez Voiture extends Vehicule : getPrixFinal() retourne prixBase + 150
13. Créez Moto extends Vehicule : getPrixFinal() applique -5% si annee < 2020, sinon prixBase
14. Créez Camionnette extends Vehicule avec $chargeUtile (en kg) : getPrixFinal() ajoute chargeUtile * 0.10 comme malus écologique
15. Implémentez getDescription() dans chaque sous-classe avec un format propre à chaque type
16. Créez un tableau $catalogue avec 2 voitures, 1 moto et 1 camionnette
17. Parcourez le catalogue avec foreach et affichez getDescription() + getPrixFinal() pour chaque véhicule
18. Calculez et affichez le prix moyen du catalogue (somme des getPrixFinal() / nombre de véhicules)

🚀 **Bonus :**

c. Ajoutez une méthode statique Vehicule::getMostExpensive(array $vehicules) qui retourne le véhicule le plus cher
d. Ajoutez une méthode isRecent() dans Vehicule qui retourne true si annee >= (année courante - 3)